Type-checking in LAMBDA

How the implementation represents and checks types

CS 413

October 20, 2026

What this implementation checks

- Source annotations give every bound variable a type.
- The parser stores those annotations in the expression tree.
- A recursive checker walks the tree and checks each operation.
- A small unifier connects unknown element types for empty lists and options.

The central question

Given the expression tree and the types of names in scope, can every part of this expression be used consistently?

Types are written at binders

```
let
  inc : int -> int =
    lam (n : int) => n + 1
  answer : int = inc(41)
in
  answer
end
```

The annotation on `inc` describes its whole function type. The lambda annotation gives the parameter type. The let annotation states what the initializer must produce.

Each binder is explicit, including recursive function names and their parameters.

The type vocabulary

$$T ::= \text{int} \mid \text{bool} \mid T \to T \mid (T_1, \ldots, T_n)$$
$$\mid \text{List}[T] \mid \text{Option}[T]$$

- Function: `int -> bool`
- Tuple: `(int, bool)`
- Empty tuple: `unit`
- List and option: `List[int]`, `Option[bool]`

Lists and options are homogeneous: every element or wrapped value has the same type.

Types have matching data structures

The checker represents types as Python objects:

Type object	Meaning
`TInt()`	integer
`TBool()`	boolean
`TFun(argument, result)`	function type
`TTuple(items)`	tuple with component types
`TList(element)`	homogeneous list
`TOption(element)`	option value
`TMeta(id)`	temporary unknown type

For example, `int -> List[bool]` is `TFun(TInt(), TList(TBool()))`.

The AST keeps each annotation

The parser turns source into dataclass nodes. The checker reads their fields.

Lambda

`DOElam(arg1, arg2, ann)`

- `arg1`: parameter name
- `arg2`: body
- `ann`: parameter type

Recursive function

`DOEfix(arg1, arg2, arg3, ann, parameter_type)`

- function name and parameter name
- body
- declared function and parameter types

Let declarations similarly store their declared type alongside the initializer.

Parsing annotations

The parser handles types with a small recursive-descent routine:

1. Read a type atom: `int`, `bool`, a tuple, `List[T]`, or `Option[T]`.
2. If an arrow follows, parse the result type recursively.
3. Build a `TFun` node. Recursion makes arrows associate right.

Right-associative arrows

`int -> bool -> int` means `int -> (bool -> int)`.

Malformed syntax, including a missing annotation, is rejected before type-checking begins.

The environment is a type lookup table

$$\Gamma = \{\texttt{inc} : \text{int} \to \text{int}, \quad \texttt{xs} : \text{List[int]}\}$$

- Checking a variable looks up its name in Γ.
- Checking a lambda adds its parameter and annotation for the body.
- Checking a let initializer uses the current environment.
- After the initializer checks, the new name and declared type enter the environment for later declarations.

Sequential scope

In `let x : int = 4 y : int = x + 1 in y end`, the first `x` is available in the initializer for `y`.

The checker follows the expression tree

The nested `check(expr, env)` function handles one AST form at a time.

Examples

- integer literal returns `TInt`
- variable returns its environment type
- addition checks two integers
- comparison checks two integers and returns boolean

Conditional

$$\frac{c : \text{bool} \quad a : T \quad b : T}{\text{if } c \text{ then } a \text{ else } b : T}$$

The implementation checks the condition, checks both branches, then unifies their types.

Function application creates a type constraint

For an application `f(arg)`, the checker:

1. checks the type of `f`
2. checks the type of `arg`
3. requires `f` to have type `argumentType -> resultType`

A valid call

If `inc : int -> int`, then `inc(41) : int`.

A rejected call

`inc(true)` creates the constraint `int = bool`, which cannot be solved.

Unification fills in unknown element types

`list_nil()` has no element to inspect, so the checker gives it a fresh type variable, say α.

$$\texttt{list_nil()} : \text{List}[\alpha]$$

In `list_cons(4, list_nil())`, the integer head constrains α to `int`. The result is `List[int]`.

- `TMeta(id)` represents a fresh unknown.
- Unification records substitutions from unknowns to known types.
- An occurs check prevents circular types such as a type being a list of itself.

The same mechanism handles `optn_nil()` and the result of `list_head`.

Checking recursion

```
fact : int -> int =
  fix fact(n: int): int =>
    if n <= 1 then 1 else n * fact(n - 1)
```

The checker verifies three things:

1. The recursive name `fact` has the declared type `int -> int` inside its body.
2. The parameter `n` has type `int`.
3. The body has the declared result type `int`.

A recursive call is checked like any other function application.

A worked example from the test programs

```
apply_twice : (int -> int, int) -> int =
  lam (p : (int -> int, int)) => p.0(p.0(p.1))
```

The environment inside the body contains:

$$\texttt{p} : (\text{int} \to \text{int}, \text{int})$$

So projection `p.0` has type `int -> int` and `p.1` has type `int`. Applying the function twice still returns `int`, which matches the annotation.

Type errors point to the violated constraint

The examples in `mytest/type_error_*.lam` isolate common failures:

- `is_small(true)`: expected integer argument, got boolean.
- `if 1 then ...`: condition must be boolean.
- `if true then 10 else false`: branch types disagree.
- `list_cons(true, list_cons(1, ...))`: list elements disagree.
- Recursive body returns boolean where its annotation promises integer.

The exception is `TypeCheckError`, a subtype of Python's `TypeError`.

Using the checker

```
from pathlib import Path
from lambda2 import d0cls_parse, d0cls_typecheck
from lambda2 import d0exp_parse_and_typecheck

decls = d0cls_parse(Path('mytest/examples.lam').read_text())
d0cls_typecheck(decls)

expr, typ = d0exp_parse_and_typecheck('1 + 2')
```

For a complete program, call the checker before the evaluator. The evaluator retains dynamic checks, but does not invoke the static checker automatically.

Scope and limits

- The checker is monomorphic: each let-bound name has one type.
- Bound variables are annotated; there is no type inference for parameters.
- Empty list and option types are solved from their uses.
- `list_head`, `optn_get`, and division can still fail at runtime on empty values or a zero divisor.
- Type-checking does not prove termination or program intent.

Takeaway

The implementation turns type rules into a recursive AST walk, with unification where an element type is initially unknown.